

Integration: Unified Pipeline and Token Injection

Architecture Overview I

The three resource layers described in @sec:pools, @sec:rules, and @sec:tools are orchestrated by a single function in src/integration.py. The run_integration_demo() function calls all three subsystems in a defined order, collects their results into a structured dictionary, and writes summary counts to output/data/manuscript_variables.json for injection into this manuscript at render time. @fig:architecture illustrates the complete architecture.

```
run_integration_demo()
    read_bibliography_fond()           → {"manifest": ..., "bi
    read_contacts_fond()               → {"manifest": ..., "co
    read_datasets_fond()               → {"manifest": ..., "da
    validate_against_rules("template_project_rules") →
    validate_against_rules("template_manuscript_rules") →
    discover_tools()                   → [{"name": ..., "manif
    validate_tool_scripts_exist(<each tool>)           →
```

Architecture Overview II

The function returns a top-level dict with keys `fonds`, `rules`, `tools`, and `summary`. The `summary` sub-dict provides the counts that populate manuscript tokens.

Manuscript Variable Tokens I

The token injection system bridges the integration pipeline and the manuscript prose. Tokens use double-brace syntax:

`{{integration.fonds_loaded}}` expands to the integer count of fonds successfully loaded during the most recent integration run. Tokens are resolved by `scripts/03_generate_manuscript.py`, which reads `output/data/manuscript_variables.json` and substitutes each token before the manuscript is passed to the rendering engine.

The current `manuscript_variables.json` contains the following summary values (see @fig:counts for a visual representation):

Token	Value
<code>{{integration.fonds_loaded}}</code>	<code>{{integration.fonds_loaded}}</code>
<code>{{integration.rules_sets_ok}}</code>	<code>{{integration.rules_sets_ok}}</code>
<code>{{integration.tools_discovered}}</code>	<code>{{integration.tools_discovered}}</code>
<code>{{integration.bib_entries}}</code>	<code>{{integration.bib_entries}}</code>

Manuscript Variable Tokens II

This table is itself token-injected: the values shown are those produced by the pipeline, not hard-coded by the manuscript author. If the pipeline results change — for example, because a new fond is added — re-running `scripts/03_generate_manuscript.py` updates the manuscript automatically, without manual editing. This property is central to reproducibility: the manuscript's quantitative claims are always consistent with the code that generated them [Stodden2016enhancing].

Methods: The Script Pipeline I

Three thin orchestration scripts govern the integration workflow (@fig:pipeline):

Script	Purpose	Key output
scripts/01_validate_sources.py	Validate presence and well-formedness of all resources	Console report
scripts/02_run_integration.py	Run <code>run_integration_demo()</code> and print JSON summary	Console JSON
scripts/03_generate_manuscript.py	Write output/data/manuscript_variables.json	JSON file

Methods: The Script Pipeline II

Each script is ~50 lines, imports all business logic from `src/`, and uses `argparse` for optional flags. This thin-orchestrator pattern [Wilson2014best] ensures that all testable logic is in `src/` at 90% coverage, while the scripts themselves remain readable without a test suite.

Resilience Design I

The integration layer enforces resilience at three levels, corresponding to the three failure modes a monorepo integration pipeline encounters:

1. **Resource absence:** A fond, rule set, or tool directory may not yet exist if the resource was created by a parallel agent that has not yet completed. All three reader modules return `None` or empty collections in this case, and `run_integration_demo()` reports the missing resource in the summary dict without raising.
2. **Schema malformation:** A manifest may be present but contain invalid YAML or missing required fields. Readers catch `yaml.YAMLError` and return a degraded result (`status="partial"` in rule validation; `None` in fond reading) rather than propagating the parse error.

Resilience Design II

3. **Script absence:** A tool may declare entrypoints in its manifest that have not yet been created.
`validate_tool_scripts_exist()` detects this at discovery time and returns `status="partial"` with a list of missing script paths, so the pipeline can report the defect without attempting to invoke a non-existent script.

This three-level resilience design reflects the principle that shared-resource pipelines should fail *informatively* rather than *catastrophically* — surfacing the cause of incompleteness in structured output that downstream consumers can act on [Taschuk2017ten].

Test Coverage I

The four `src/` modules are covered by 38 tests across four test files in `tests/`. Tests use real file paths, real YAML files, and real BibTeX content rather than mocks, ensuring that coverage numbers reflect genuine code paths through the resource-discovery logic. The current coverage report shows 90% line coverage for all four modules. The `tests/test_integration.py` suite includes an end-to-end test that calls `run_integration_demo()` and asserts that the `summary` dict contains the expected keys with non-negative integer values — a contract test that verifies the token injection pipeline's data source.